

THE UNLOX ACADEMY

Weekly Assessment

Joins, Subqueries Windows & CTEs Weekly Assessment

*DA / DS Track***Total marks: 100 Questions: 40****INSTRUCTIONS**

Complete all 40 questions.

The assessment uses a fresh 3-table dataset called bookmark. Run bookmark_setup.sql in MySQL Workbench BEFORE starting the assessment.

Section A (Theory, 8Q × 2.5) - write your answer letter (a/b/c/d).

Section B (Output Prediction, 8Q × 2.5) - describe what the query returns (row count, values).

Section C (Applied SQL, 24Q × 2.5) - write the actual SQL query. Formatting counts.

You may use Workbench to test your queries. You may NOT use ChatGPT / Copilot / any AI assistant. Any AI-detected answer receives zero for the entire section.

Submit as one .sql file or Word doc. File name: Week4_Assessment_<YourName>.

The Dataset - bookmark

A fictional Indian online bookstore. Three related tables:

Table	Rows	Key columns
authors	10	author_id (PK), name, country, born_year, mentor_id (FK → authors)
books	25	book_id (PK), title, author_id (FK), genre, price, published_year, avg_rating
sales	40	sale_id (PK), book_id (FK), quantity, sale_date, city, customer_type

Key data properties (helps you sanity-check queries)

- authors have 3 mentor relationships** (via mentor_id) — Preeti→Chetan, Alex→Yuval, Malcolm→Yuval. The rest have NULL mentor_id.
- Only 10 of the 25 books have been sold** — 15 books have never appeared in a sale (great LEFT JOIN + IS NULL / NOT EXISTS material).

- **5 Indian authors, 5 international** (USA, UK, Israel, Canada). 6 distinct genres.
- **Every author has at least one book** — so LEFT JOIN authors→books returns 25 rows (never 26).
- **One book has NULL avg_rating** (The Wager — new 2024 release).

Sample rows

authors:

1	Chetan Bhagat	India	1974	NULL	
4	Preeti Shenoy	India	1971	1	< - mentored by Chetan
7	Yuval Harari	Israel	1976	NULL	
10	Malcolm Gladwell	Canada	1963	7	< - mentored by Yuval

books:

115	Atomic Habits	author 6	Self-Help	349.00	2018	4.70
121	Rich Dad Poor Dad	author 9	Business	299.00	2000	4.40
125	The Wager	author 7	History	599.00	2024	NULL

sales:

1003	book 115	qty 5	2024-01-22	Bengaluru	Corporate
1004	book 116	qty 4	2024-01-25	Delhi	Retail

Section A - Theory

Choose the correct option (a, b, c, or d) for each question. Write only the letter.

A1. What is the key difference between INNER JOIN and LEFT JOIN?

- a) INNER JOIN always returns more rows than LEFT JOIN
- b) INNER JOIN returns only matched rows; LEFT JOIN returns all left-table rows with NULL for non-matches
- c) LEFT JOIN can only be applied to the primary table
- d) They produce identical results in all cases

Your answer: _____

A2. What does OVER (PARTITION BY department) do in a window function?

- a) Filters rows to only one department
- b) Groups rows by department and collapses them into one row per group
- c) Restarts the aggregate calculation for each department while keeping all input rows
- d) Sorts the result by department

Your answer: _____

A3. Why is NATURAL JOIN considered a production anti-pattern?

- a) MySQL doesn't support it
- b) It's always slower than INNER JOIN
- c) It silently joins on ALL columns with matching names — behaviour can change unexpectedly when the schema is modified
- d) It always produces duplicate rows

Your answer: _____

A4. When should you prefer UNION ALL over UNION?

- a) When you specifically want to remove duplicates
- b) When the two queries cannot produce overlapping rows, OR when performance matters more than deduplication
- c) When you have exactly two SELECT statements
- d) When the two queries return the same table

Your answer: _____

A5. What is the NOT IN + NULL trap?

- a) NOT IN cannot be used with VARCHAR columns
- b) If the subquery inside NOT IN returns any NULL value, the entire NOT IN silently returns zero rows

- c) NOT IN is always slower than NOT EXISTS
- d) NOT IN only works with numeric columns

Your answer: _____

A6. What defines a correlated subquery?

- a) A subquery that runs once before the outer query
- b) The inner query references a column from the outer query, causing it to re-execute for each outer row
- c) A subquery that returns exactly one row
- d) A subquery nested inside another subquery

Your answer: _____

A7. Why can't you use WHERE with a window function alias directly?

- a) Window functions cannot produce aliases
- b) WHERE runs BEFORE window functions are calculated, so the alias doesn't exist yet
- c) Aliases work only in the SELECT clause
- d) This is a MySQL-specific bug

Your answer: _____

A8. What is a CTE (Common Table Expression)?

- a) A permanent table stored in the database
- b) A named temporary result set defined with the WITH clause, existing only for the duration of the query
- c) A syntax for creating indexes
- d) A special type of JOIN operation

Your answer: _____

Section B - Output Prediction

For each query below, describe what it returns. Give the row count and specific values where asked.

B1. How many rows does this query return?

```
SELECT * FROM books b
INNER JOIN authors a ON b.author_id = a.author_id;
```

Your answer:

B2. How many rows does this query return?

```
SELECT a.name FROM authors a
LEFT JOIN books b ON a.author_id = b.author_id
WHERE b.book_id IS NULL;
```

Your answer:

B3. How many rows does this self-join return, and who are they?

```
SELECT a.name AS author, m.name AS mentor
FROM authors a
JOIN authors m ON a.mentor_id = m.author_id;
```

Your answer:

B4. How many rows does this UNION return?

```
SELECT name FROM authors WHERE country = 'India'
UNION
SELECT a.name FROM authors a
JOIN books b ON a.author_id = b.author_id
WHERE b.genre = 'Mythology';
```

Your answer:

B5. What single value does this query return?

```
SELECT COUNT(*) FROM books
WHERE price > (SELECT AVG(price) FROM books);
```

Your answer:

B6. How many rows does this query return?

```
SELECT * FROM books b
WHERE NOT EXISTS (
    SELECT 1 FROM sales s WHERE s.book_id = b.book_id
);
```

Your answer:

B7. What is the total_qty of the top-selling book (first row)?

```
SELECT b.title, SUM(s.quantity) AS total_qty
FROM sales s JOIN books b ON s.book_id = b.book_id
GROUP BY b.book_id
ORDER BY total_qty DESC
LIMIT 1;
```

Your answer:

B8. What does this query output for Business genre books?

```
SELECT title, price,  
       RANK() OVER (PARTITION BY genre ORDER BY price DESC) AS rank_in_genre  
FROM books WHERE genre = 'Business';
```

Your answer:

Section C - Applied SQL

Write the actual SQL query for each question. Formatting counts - uppercase for keywords, align clauses on new lines for complex queries.

C1 - Basic Joins (INNER, LEFT, aggregate)

C1. Write a query using INNER JOIN to display each book's title alongside its author's name.

Your query:

C2. Write a LEFT JOIN query listing every author's name and every book they've written. Authors with no books should still appear (once) with NULL for book columns.

Your query:

C3. Write a query to compute total revenue (sum of quantity × price) per genre. Sort by revenue descending.

Your query:

C4. Write a query to find the single city that has generated the most revenue.

Your query:

C2 - Extended Joins (RIGHT, FULL OUTER, SELF)

C5. Write a RIGHT JOIN query showing every book (via RIGHT JOIN from authors) alongside its author. Explain briefly why the result count matches INNER JOIN here.

Your query:

C6. Write a FULL OUTER JOIN of authors and books using the UNION trick (since MySQL doesn't support FULL OUTER directly).

Your query:

C7. Write a SELF JOIN query showing the name of every mentored author alongside their mentor's name.

Your query:

C3 - Set Operations (CROSS, UNION, anti-join)

C8. Write a CROSS JOIN query generating every possible combination of city and customer_type from the sales table (use SELECT DISTINCT subqueries).

Your query:

C9. Write a UNION query combining: (a) all Indian authors, (b) all authors born after 1970. Result should be deduplicated.

Your query:

C10. Write an anti-join query using LEFT JOIN + IS NULL to find all books that have NEVER been sold.

Your query:

C4 - Subqueries (scalar, IN, ANY/ALL, correlated)

C11. Write a scalar subquery to find all books priced above the overall average book price.

Your query:

C12. Write a query using IN with a subquery to show all sales of books whose genre is either 'History' or 'Mythology'.

Your query:

C13. Write a query using > ALL to find books priced higher than every single Fiction book.

Your query:

C14. Write a correlated subquery to find books priced above their genre's average price.

Your query:

C5 - EXISTS / NOT EXISTS

C15. Write an EXISTS query to find all authors who have written at least one book published after 2018.

Your query:

C16. Write a NOT EXISTS query to find all authors who have never written a book in the 'Business' genre.

Your query:

C17. Write a query using IN with a subquery to find all sales made for books written by Indian authors.

Your query:

C6 - Window Functions (PARTITION, ROW_NUMBER, LAG, running total)

C18. Write a query showing every book alongside its genre's average price (using AVG OVER PARTITION BY) - all 25 rows should appear.

Your query:

C19. Write a query using ROW_NUMBER to find the top 2 highest-priced books in each genre.

Your query:

C20. Using LAG, write a query showing each sale of book_id 115 (Atomic Habits) alongside the quantity of the previous sale (sorted by sale_date). The first sale should show NULL for previous quantity.

Your query:

C21. Write a query using SUM() OVER (ORDER BY sale_date) to compute a running total of quantity across all sales.

Your query:

C7 - CTEs & Synthesis

C22. Rewrite this query using a CTE (WITH clause) for readability. Compute total quantity sold per book, then join to book details showing title and total.

Your query:

C23. Write a multi-CTE query: first CTE computes revenue per genre, second CTE ranks genres by revenue using RANK(). Final SELECT shows genre, revenue, and rank.

Your query:

C24. Write a comprehensive query: for each genre, show the top-selling book (by total quantity sold), its author's name, its total quantity, and the genre's total revenue. Use CTEs + window functions + joins. Sort by genre revenue descending.

Your query:

Submission

- **File name:** Week4_Assessment_<YourName>.sql (or .docx)
- **Upload to:** The Google Form. Deadline as announced.
- **Integrity:** Your work only. AI assistance, copy-paste from classmates, or tutorial-site answers = zero and re-submission required.

All the best.
— The Unlox Academy